Timers

Chapter Contents

Overview

In this chapter, we show how timer peripherals work to measure elapsed time, count events, generate events at specified times, help the processor recover from an out-of-control program, and perform other more advanced features. Using timers, it is also possible to output a periodic digital signal with controllable frequency and duty cycle. We will cover the concepts behind these features and how to use them.

Concepts

The core of a timer or ***timer/counter*** peripheral is a digital ***counter*** whose value changes by one each time the counter is clocked. The faster the clocking rate, the faster the device counts. If the timer's input clock frequency is 10 MHz, then its period is the inverse of 10 MHz: 1/(10 MHz) or 0.1 µs. Hence one count (increment or decrement) of the register represents 0.1 µs. We can measure how much time has passed since the counter was reset by reading the counter value and multiplying it by 0.1 µs. For example, if the counter value is 15821, and the count direction is up (incrementing), then we know that 1582.1 µs have passed since the counter was reset.

> **timer/counter**
> *Peripheral which measures time or counts events*

> **counter**
> *Digital circuit which counts number of input pulses*

Timer Circuit Hardware

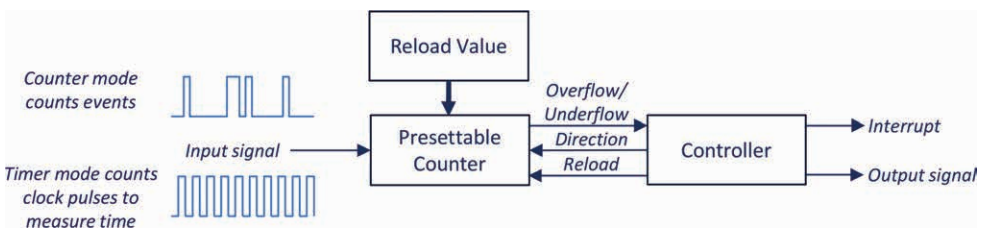

Figure 7.1 Timer peripheral hardware is built around a counter.

Figure 7.1 shows the block diagram of a basic timer peripheral's hardware. A transition on the input signal may represent either an event or a fixed time interval. Regardless, the transition causes the counter to change (e.g. increment by one). The timer peripheral can therefore count events or measure time. Other hardware is often added to make it even more flexible. Timer peripherals are typically able to measure elapsed time, count events, generate events at fixed times, generate waveforms, or measure pulse widths and frequencies. This circuitry controls factors such as:

- Which signal source it counts. If it counts a signal with a known frequency, then we can use it to measure elapsed time
- When it starts and stops running
- Whether it counts rising or falling edges
- Which direction it counts
- What happens when it overflows
- If and how it is reloaded
- Whether its value is captured by another register
- Whether it generates a signal or an interrupt

Example Timer Uses

Periodic Timer Tick

One of the most basic uses of a timer peripheral is to generate a periodic interrupt event. The time between interrupts is steady because it is controlled by hardware and not affected by software delays. We may want to monitor elapsed time, for example, to generate a timestamp for logged events. The timer's counter register measures the current time with a high resolution. If needed, we can extend the timer's range by using its overflow interrupt to trigger an interrupt service routine that increments an overflow counter.

Many software tasks in embedded systems require time delays. A task could use a busy-waiting time delay loop to wait, but this does not share the processor with other tasks. A periodic timer interrupt can be used instead to track the time delay and start the task after it has elapsed.

Watchdog Timer

It is difficult to make a program for an embedded system completely perfect. One reason is that developers sometimes translate their ideas into code (and peripheral configurations) incorrectly, introducing bugs. This may be from misunderstanding what a C code statement really means, how a peripheral really operates, or how different parts of a system might interact (e.g. preemption). Another reason is that the developer has translated an imperfect idea to code. It is difficult for humans to imagine all possible sequences of combinations of inputs to an embedded system, so we often leave some of these out of our specification of how the system should behave, and don't consider them when designing the system to meet that specification.

We can reduce the number of both types of bug with various methods (e.g. testing, rigorous design process, design reviews), but eliminating all bugs will be expensive and probably infeasible. So bugs are present essentially in all embedded system software. We still want the system to operate correctly most of the time. One way to do this is to restart the program automatically if an error is detected.

A ***watchdog timer (WDT)*** is a peripheral that tries to detect if the program goes out of control, in which case the WDT resets the processor to restart the program. The WDT uses a counter to keep track of the elapsed time and expects to be signaled (serviced) by the program periodically, such as once per second. If the WDT is not serviced within the expected time, then it will reset the processor. If the WDT is serviced, then it will reset its counter to begin a new time measurement.

> **watchdog timer (WDT)**
> *Hardware peripheral used to reset out-of-control program*

The program is responsible for servicing the WDT at correct times. There are many types of bugs in the program that can keep it from timely WDT servicing, making this a useful error detection method. Nearly all MCUs provide a WDT, and good embedded systems use them. However, bugs that do not affect the timing of the WDT servicing will not be detected.

Time and Frequency Measurement

We can measure a signal's frequency or period with a timer peripheral. For example, an anemometer generates a pulse signal with a wind-dependent frequency. The anemometer in Figure 7.2 has a rotating section with three arms and cups and one or more magnets mounted near the axle. A magnetic sensor (such as a reed switch) is mounted on the fixed mast. The magnetic sensor generates a pulse each time a magnet passes by, so the frequency of the signal $\boldsymbol{f_{anem}}$ will be proportional to the wind speed, neglecting the errors caused by inertia and friction. We can then calculate the wind speed $\boldsymbol{v_{wind}}$ based on the $\boldsymbol{f_{anem}}$ and the distance $\boldsymbol{r}$ of the anemometer cups from the axis:

$$v_{\text{wind}} \approx 2{*}\pi{*}r{*}\, f_{\text{anem}}$$

Figure 7.2 Cup anemometer used to measure wind speed. Photo by author.

How do we find the signal's frequency or period (the inverse of the period)?

We can measure the signal's frequency by counting how many pulses have occurred during a fixed time period. We configure the peripheral in event counter mode. To make a measurement, we clear the timer, start it running, wait for the fixed measurement time, and then read the counter value. Dividing the count value by the measurement time gives the signal frequency, which we then scale to provide wind speed.

We can measure the signal's period by measuring the time between successive rising edges. We configure the peripheral in timer mode, so it counts at a fixed rate. We then start the timer running. When the input signal has a rising edge on the input signal, we capture the value of the timer's counter with an interrupt service routine. We then wait for the next rising edge and capture the new value of the timer's counter. The period of the signal is equal to the difference in the count values divided by the count frequency. We invert the period and then scale it to determine wind speed.

Timer peripherals typically have additional support circuitry to simplify these measurement procedures to improve accuracy and reduce software processing and complexity, as we will see shortly.

PWM Signal Generation

Pulse-width modulation (PWM) is a method to send more than one bit of information on a single digital signal line. Rather than encode the information serially as a series of bits (as we will see in the next chapter), the information is encoded as the fraction of time that the signal is a logic one (called the ***duty cycle***). Because the signal is sent in a digital format, it is much less vulnerable to electrical noise.

Pulse-width modulation (PWM)
Method for encoding information onto a single digital signal based on duty cycle

Duty cycle
Fraction of time that a digital signal is asserted

Some devices can be driven by a PWM signal or a buffered version that can provide more power, voltage, and current. For example, a buffered PWM signal may drive a motor at a reduced speed or partially dim a light. The high-frequency components of the signal are averaged by the inertia of the motor or the persistence of human vision. A PWM signal can be averaged with a low-pass filter to create an analog voltage if we do not have a DAC.

Remember the hot-plate example from the first chapter? We can use PWM to control the heating element. Rather than being only fully on or fully off, the heating element can take on intermediate heating values, proportional to the duty cycle. This will enable more precise control with less error and overshoot.

Figure 7.3 shows an example of a PWM signal with a duty cycle of about 70%. There are several terms that describe PWM signal characteristics:

- The on-time T_{On} is the amount of time the signal is active.
- The off-time T_{Off} is the amount of time the signal is inactive.
- The signal frequency f indicates how many pulses are sent per second.
- The period T is the inverse of the frequency: $T=1/f$. It is also the sum of the on and off times: $T = T_{On} + T_{Off}$
- The duty cycle D is the on-time divided by the period: $D = T_{On}/T$
- The polarity of the signal may be active-high or active-low. For active-high signals, the signal is on (true) when it is a logic one. For active-low signals, the signal is on (true) when it is a logic zero.

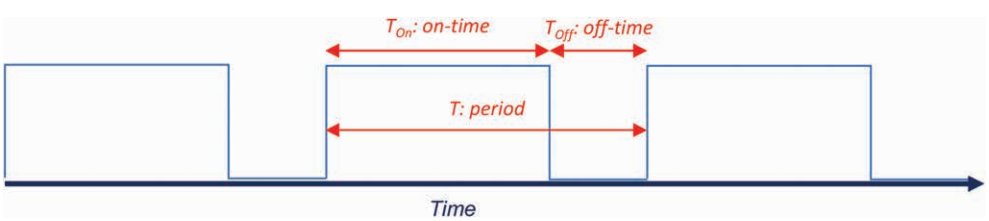

Figure 7.3 Example of pulse-width modulated signal.

Timer Peripherals

SysTick Timer

The Cortex-M0 core contains a simple timer peripheral called the SysTick timer, shown in Figure 7.4. It is designed to provide a periodic tick for system operation, such as in a scheduler or operating system kernel. Regardless of MCU device manufacturer, all Cortex-M processors (e.g. M0, M0+, M3, M4, M7, etc.) have one. This helps make the system software more portable across different devices.

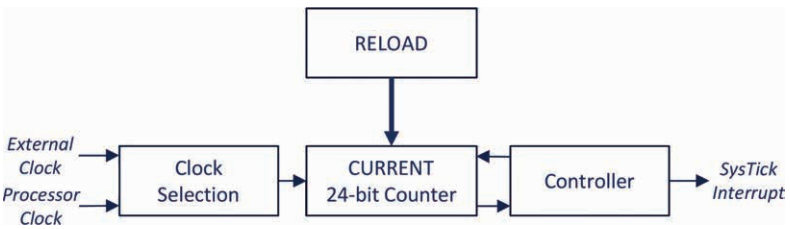

Figure 7.4 Overview of SysTick timer circuitry.

The counter in the ***SysTick timer*** is 24 bits long and decrements when clocked. Its current value can be read from the CURRENT register field, shown in Figure 7.5. When first enabled, the counter loads itself from the 24-bit RELOAD field in Figure 7.6. It then counts down with each input clock pulse. After the counter reaches zero it reloads itself with the RELOAD value and can generate a SysTick exception (if enabled). Writing anything to the SYST_CVR clears that register to zero.

> **SysTick Timer**
> *Timer peripheral available in Cortex-M CPU cores, typically used to generate periodic time tick*

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CURRENT[23:16]							
Reset									x	x	x	x	x	x	x	x
Access									rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	CURRENT[15:0]															
Reset	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
Access	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w

Figure 7.5 SysTick Timer current value register (SYST_CVR or SysTick->VAL) contains CURRENT field.

The counter divides the input frequency by a factor of RELOAD+1. In order to divide an input frequency f_{in} by a factor of N, we store N-1 in the RELOAD field.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RELOAD[23:16]							
Reset									x	x	x	x	x	x	x	x
Access									rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	RELOAD[15:0]															
Reset	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7.6 SysTick Timer reload value register (SYST_RVR or SysTick->LOAD) contains RELOAD field.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	COUNT FLAG
Reset																0
Access																r

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLK SOURCE	TICK INT	ENABLE
Reset														x	0	0
Access														rw	rw	rw

Figure 7.7 SysTick Timer control and status register (SYST_CSR or SysTick->CTRL) contains various fields.

The CTRL register shown in Figure 7.7 controls and indicates the status for the SysTick timer.

- The CLKSOURCE field selects the clock source, which can be either the processor clock (one) or an external reference clock (zero). On the STM32F091RC MCU, the processor clock runs at up to 48 MHz, and the external reference clock is the processor clock divided by 8.
- The TICKINT field controls whether counting down to zero will enable a SysTick exception request (one) or not (zero).
- The ENABLE field enables the counter when set to one.
- The COUNTFLAG field returns a one if the timer has counted down to zero since the last time this register was read. Reading SYST_CSR clears COUNTFLAG to zero, as does writing any value to SYST_CVR.

The SYST_CALIB register provides support for calibrating the timer and is not discussed further here. Further information on the SysTick timer can be found in the Arm documentation and elsewhere [1] [2].

CMSIS definitions for the SysTick timer peripheral are located in the `core_cm0.h` include file. That file also defines a function `SysTick_Config` that can be used to configure the timer. The exception handler is called `SysTick_Handler`, and the exception number is `SysTick_IRQn`.

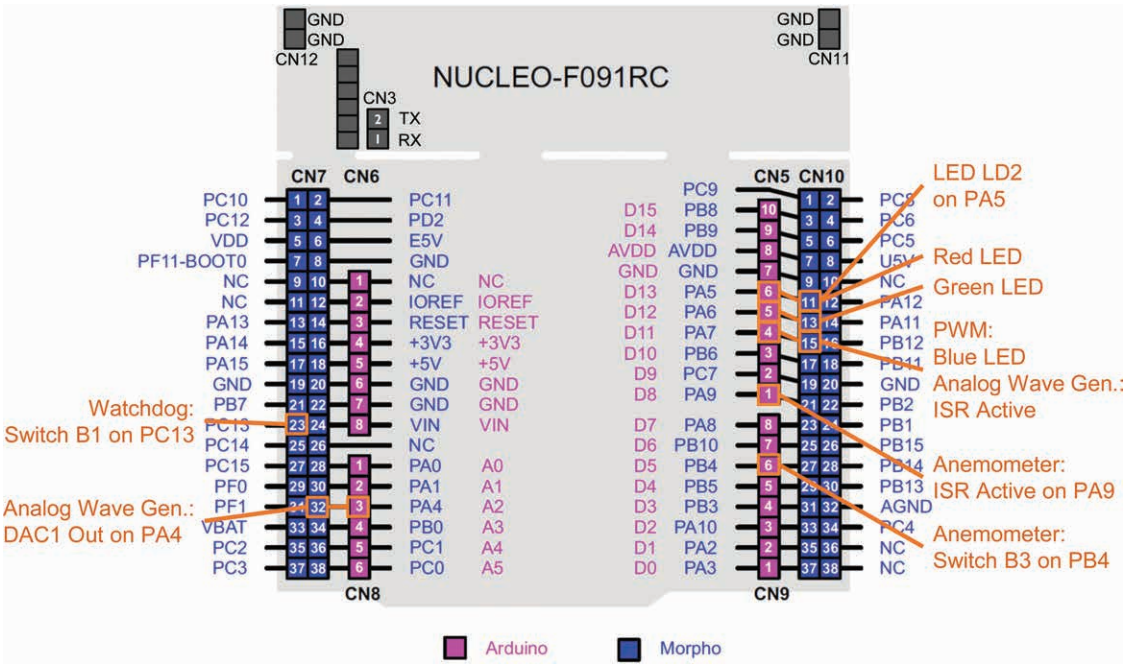

Figure 7.8 Locations of signals for timer chapter code examples.

Example: Periodic 1 Hz Interrupt

Let's configure the SysTick timer to generate an interrupt every second and change the color on the RGB LEDs. Figure 7.8 shows the signal locations for this and other example programs in this chapter. We need to divide the 48 MHz processor clock down by a factor of 48 MHz/1 Hz = 48,000,000. The value of 47,999,999 is too large to fit into the 24-bit LOAD field, so we will need to use the alternate clock source (CLKSOURCE=0), which runs at 48 MHz/8 = 6 MHz. The new division factor is 6 MHz/1 Hz = 6,000,000, and the LOAD value of 5,999,999 does fit into 24 bits.

```
#define F_SYS_CLK (48000000L)
void Init_SysTick(void) {
  // SysTick is defined in core_cm0.h
  // Set reload field to get 1 sec interrupts
  MODIFY_FIELD(SysTick->LOAD, SysTick_LOAD_RELOAD, (F_SYS_CLK/8)-1);
  // Set interrupt priority
  NVIC_SetPriority(SysTick_IRQn, 3);
  // Force load of reload value
  MODIFY_FIELD(SysTick->VAL, SysTick_VAL_CURRENT, 0);
  // Enable interrupt, enable SysTick timer
  SysTick->CTRL = SysTick_CTRL_TICKINT_Msk | SysTick_CTRL_ENABLE_Msk;
}
```

Listing 7.1 Function to initialize SysTick Timer to generate 1 Hz interrupts.

STM32F091RC Timer and PWM Module

The timer module (TIM) consists of a time-base unit (with a 16-bit or 32-bit counter) and multiple channels that use the core counter's value to measure the timing of input signals or generate output signals. Figure 7.16 shows a block diagram of the TIM circuitry. To use a TIM peripheral, enable its peripheral clock in the RCC. For example, enable TIM3's peripheral clock by setting the field TIM3EN in RCC->APB1ENR to 1.

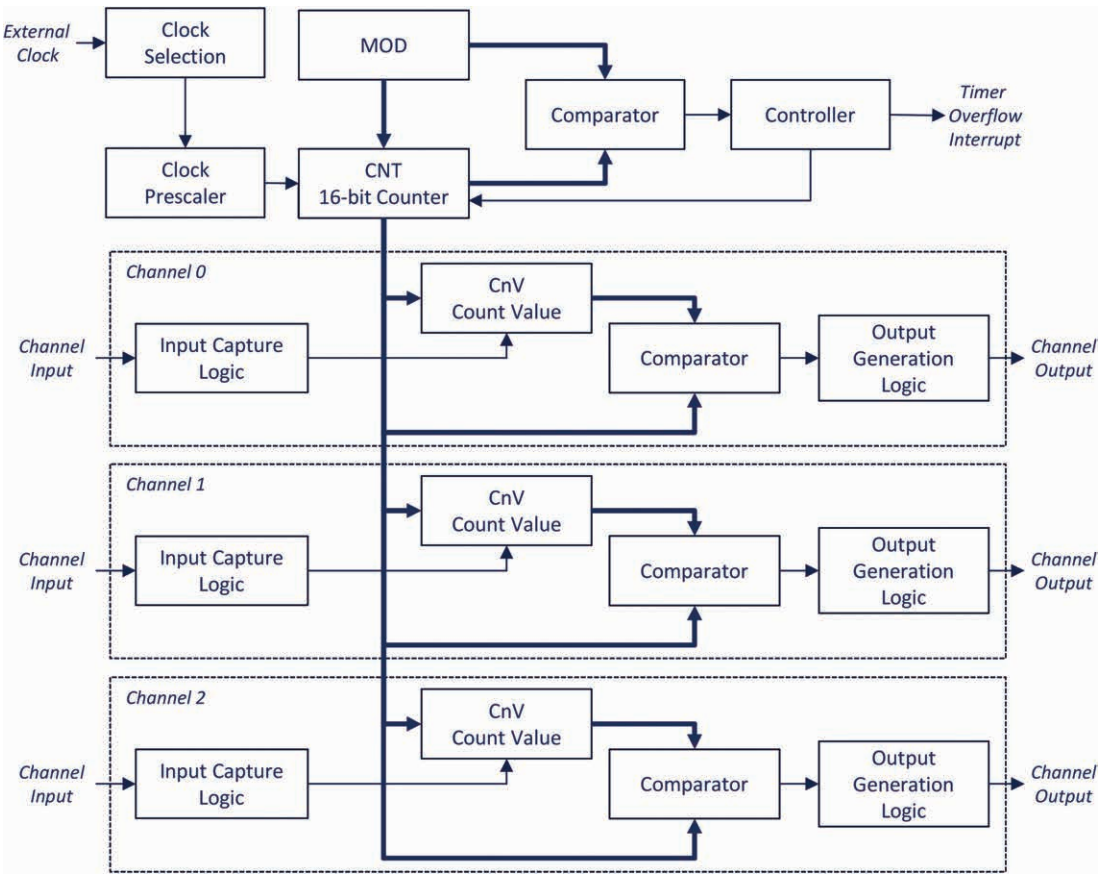

Figure 7.16 Timer peripheral with time-base unit and multiple channels for input and output processing.

The STM32F091RC MCU has 10 TIMs: one 16-bit advanced-controller timer (TIM1) with four channels, one 32-bit timer (TIM2) with four channels and seven 16-bit timers with up to four channels (TIM3, TIM6, TIM7, TIM14, TIM15, TIM16 and TIM17). Full information is available in the documentation [3] [6].

This chapter refers specifically to TIM3. Other timers available on STM32F091RC may have more or fewer features and some of the registers may differ slightly. You can refer to the reference manual for more information [3].

Time Base Unit

The time-base unit for TIM3 counts pulses from a clock input using a 16-bit up/down counter called TIMx_CNT. One of several clock input sources can counted: an internal clock (CK_INT), an external input pin, an external trigger input, or internal triggers. This chapter only discusses the internal clock, which is derived from PCLK (48 MHz in these examples). It is selected by clearing the SMS field of TIMx_SMCR to zero.

The prescaler is a hardware circuit that can reduce the number of input events which are passed to TIMx_CNT for counting. The prescaler divides down the input frequency by a factor of 1 to 65536: one plus the value in the 16-bit PSC field of the TIMx_PSC register. For example, to prescale the input clock by a factor of 8, load TIMx_PSC with 7.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	Res.	Res.	Res.	CKD		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
Reset							0	0	0	0	0	0	0	0	0	0
Access							rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7.17 TIMx_CR1 control register 1.

The TIM control register 1 (TIMx_CR1) controls the basic operation of the TIM's core and is shown in Figure 7.17. We'll start with up-counting mode, which is the most basic. Each clock signal from the prescaler causes CNT to increment, starting at zero. When TIMx_CNT counts past the limit value set in TIMx_ARR, a hardware comparator circuit detects this overflow condition and will perform one or more actions:

- Clear the counter to zero.
- Set update flag UIF (in TIMx_SR) to one (unless updates are disabled, i.e. UDIS is one).
- Generate a TIMx interrupt (unless the timer interrupt is disabled, i.e. UIE field in TIMx_DIER is zero).
- Disable the counter if one-pulse mode is selected (OPM is one). This results in just one count sequence occurring. If OPM is zero, the counter remains enabled and repeats the cycle.

Down-counting mode is similar to up-counting mode, but the counter starts at the value in TIMx_ARR and decrements. Counting down past zero causes an underflow, causing the counter to reload with TIMx_ARR and possibly setting UIF or generating a TIMx interrupt.

In up/down-counting modes, the counter counts up from zero and then down from TIMx_ARR. This is also called "center-aligned mode" because it is quite useful for pulse-width modulation (discussed later).

The counting mode is selected with the CMS and DIR fields of TIMx_CR1. To select either up-or down-counting mode, CMS must be cleared to 00, and DIR must be 0 for up-counting or 1 for down-counting. Setting CMS to 01, 10 or 11 selects one of three up/down counting modes.

After configuring the timer, be sure to set the CEN field to one to enable it. There are other fields in TIMx_CR1: the URS field indicates which source can create an event and the UDIS indicates if the events are enabled.

The TIMx will overflow or underflow at a frequency of f_{clock}/((TIMx_PSC+1)*(TIMx_ARR+1)) when in up-counting or down-counting mode, and f_{clock}/((TIMx_PSC+1)*2*TIMx_ARR)) when in up/down-counting mode.

To use timer update interrupts, make sure updates are enabled (UDIS in TIMx_CR1 is zero), interrupts are enabled (UIE in TIMx_DIER is one), and the NVIC has been configured. The TIM3_IRQHandler services the interrupt timer 3. The TIMx_SR register, shown in Figure 7.18, indicates the status of the TIM time-base and its channels. The ISR must examine the flags in this register to identify the cause of the interrupt, service it and clear that flag. Note that these flags are cleared by writing zero to them, as indicated by the "rw0" notation in the figure.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	CC4OF	CC3OF	CC2OF	CC1OF	Res.	Res.	TIF	Res.	CC4IF	CC3IF	CC2IF	CC1IF	UIF
Reset				0	0	0	0			0		0	0	0	0	0
Access				rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Figure 7.18 TIMx_SR register indicates if counter has overflowed and if any channel events have occurred.

Example: Analog Waveform Generation

We can use the timer to generate a regular interrupt in order to generate a waveform with precise timing. Listing 7.7 shows the DAC-based triangle waveform generator function from the previous chapter.

```
#define MAX_DAC_CODE 4095

void Triangle_Output(void) {
  int i = 0;
  int change = 1;

  while (1) {
    DAC->DHR12R1 = i;
    i += change;
    if (i <= 0)
      change = 1;
    else if (i >= MAX_DAC_CODE - 1)
      change = -1;
  }
}
```

Listing 7.7 Simple triangle waveform generator function from previous chapter has timing and processor sharing limitations.

There are two major limitations to this approach. First, the program's structure makes it difficult to share the processor's time with other processing activities. The function `Triangle_Output` uses an infinite loop and never completes. Second, the program's timing behavior is very fragile. On some loop iterations the conditional code in an if statement will be executed (e.g., `if (i == DAC_RESOLUTION – 1)`, `change = –1`), taking an additional amount of time. The DAC playback of a sample following such an iteration will be delayed, distorting the

signal. Adding any other processing will slow down the waveform generator, delaying its sample generation and distorting the output signal. Changing compiler settings will probably change the timing of the sample playback. These changes will force the developer to adjust the time delay in the loop.

We can eliminate this timing variability and also simplify processor time sharing by using a periodic interrupt to update the DAC at regular intervals. The ISR operates asynchronously from the main program, improving the timing stability. Our waveform generation code is short and simple enough to embed within the ISR, as shown in Listing 7.8.

```
void TIM3_IRQHandler(void){
  static int change = STEP_SIZE;
  static uint16_t out_data = 0;

  // Debug signal : Entering ISR
  GPIOA->BSRR = DEBUG_ON_MSK;
  // Clear interrupt request flag
  TIM3->SR = ~TIM_SR_UIF;
  // Do ISR work
  out_data += change;
  DAC->DHR12R1 = out_data;
  if (out_data < STEP_SIZE)
    change = STEP_SIZE;
  else if (out_data >= DAC_RESOLUTION - 1 - STEP_SIZE)
    change = -STEP_SIZE;
  // Debug signal: exiting ISR
  GPIOA->BSRR = DEBUG_OFF_MSK;
}
```

Listing 7.8 Interrupt service routine generates waveform using DAC.

We have modified the code to change the output data by `STEP_SIZE` rather than just one, making the code more configurable. In this example `STEP_SIZE` is 16 and `DAC_RESOLUTION` is 4096.

We have also added two lines of code to set GPIO A bit 7 upon entering the ISR and to clear it upon exiting. We can use an oscilloscope to monitor this signal (please refer to Figure 7.8 for locations) to determine when the processor is executing the ISR. Remember that there is additional minor time overhead for the CPU to respond to the interrupt before the ISR begins executing.

Listing 7.9 shows the code that initializes the TIM3 to generate an interrupt at a frequency of `F_TIM_OVERFLOW` (100 kHz here), given an input clock rate of `F_TIM_CLOCK` (48 MHz). We use the prescaler to divide the input frequency by a factor of `TIM_PRESCALER` (32).

```
void Init_TIM(void) {
  RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;

  TIM3->ARR = F_TIM_CLOCK / (F_TIM_OVERFLOW * TIM_PRESCALER) - 1;
  TIM3->SMCR = 0;
  TIM3->CR1 = 0; // count up
```

```
  TIM3->CR2 = 0;
  TIM3->EGR = 0;
  TIM3->PSC = TIM_PRESCALER - 1;
  TIM3->DIER = TIM_DIER_UIE; // enable update interrupt

  NVIC_SetPriority(TIM3_IRQn, 128);
  NVIC_ClearPendingIRQ(TIM3_IRQn);
  NVIC_EnableIRQ(TIM3_IRQn);

  TIM3->CR1 |= TIM_CR1_CEN;   // enable timer
}
```

Listing 7.9 Function to initialize TIM3 to generate periodic interrupt.

Figure 7.19 shows the output of the DAC (upper trace) and the debug signal (lower trace), which is one when the ISR is executing. The period of the DAC signal is about 5.1 ms, so the frequency is about 196 Hz. These show the system is working correctly.

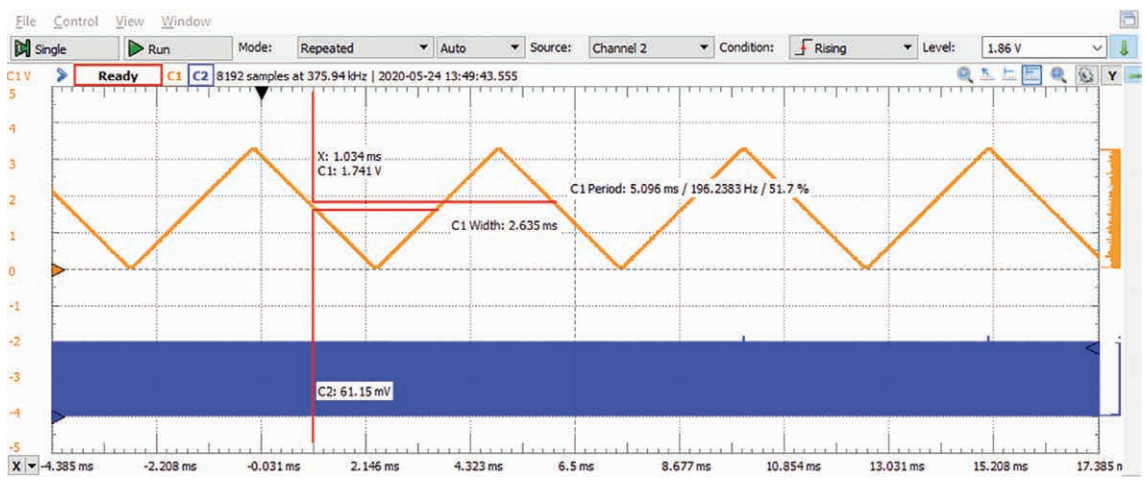

Figure 7.19 Output of ISR-based triangle waveform generator program. Upper trace is DAC output, lower trace is ISR activity (GPIO A bit 7).

Let's take a closer look at the timing of the signals. Figure 7.20 shows the DAC is updated within the ISR, as expected. The ISR takes about 1.6 µs to execute, with about 0.4 µs of overhead to enter and exit the ISR, so about 2 µs/10 µs = 20% of the CPU's time is used for the waveform generation. This leaves 80% of the CPU time for other processing, unlike the code of Listing 7.7.

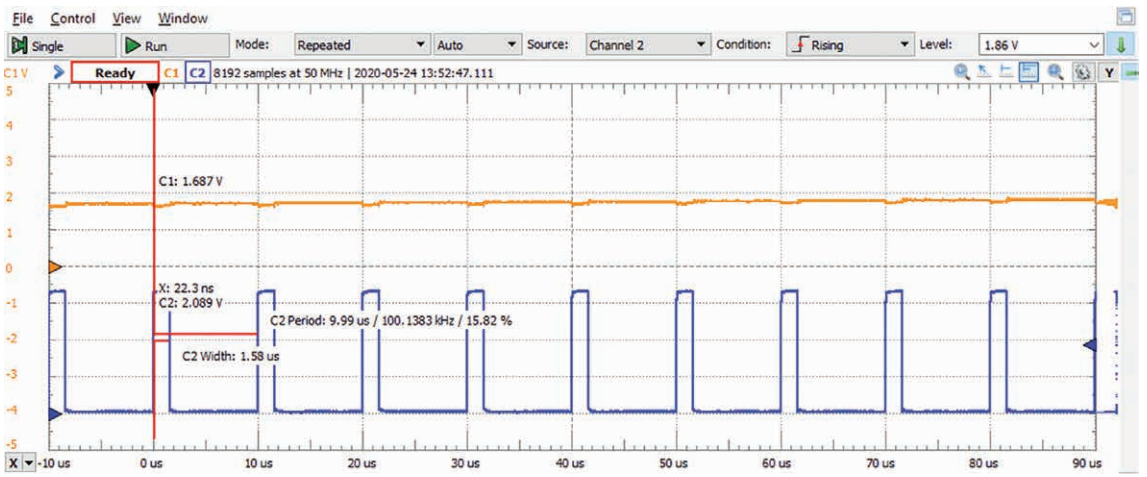

Figure 7.20 DAC output is updated every 10 µs, when ISR is active (lower trace is high). ISR is active for about 1.6 µs.

The ISR does not always take the same amount of time to execute because of the conditional code which tests if `out_data` has reached an upper or lower bound. We can see the variability of the execution time in Figure 7.21, in which the oscilloscope's infinite persistence feature is used to display all debug bit (channel 2) traces, erasing none. The ISR normally takes 1.59 µs to execute, but occasionally it takes a bit more or a bit less. Examining the ISR code again shows the DAC output is updated before this conditional code, so the DAC output waveform always will be changed on time.

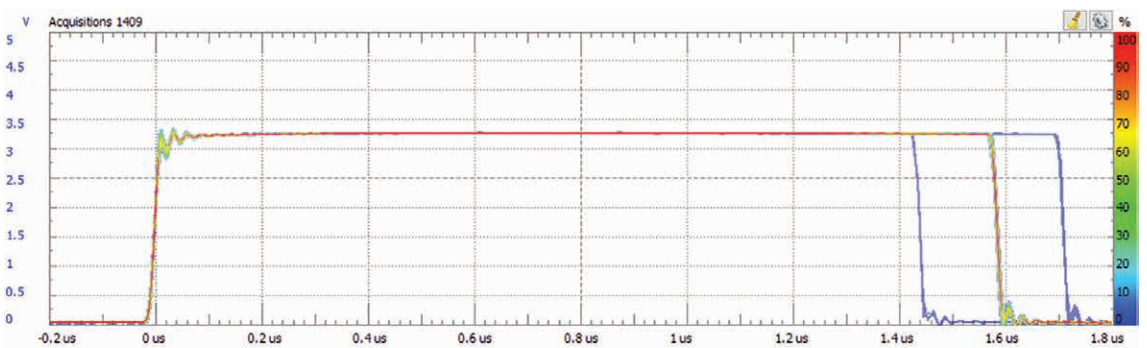

Figure 7.21 Detailed view of ISR execution time signal. ISR usually takes 1.59 µs, but occasionally takes slightly more (1.7 µs) or less time (1.44 µs). This timing variation occurs after the DAC output is updated, so timing accuracy is maintained.

TIM Channels

Each timer has multiple channels which can be used independently to monitor inputs or control outputs. Each channel y has a data register for the capture/compare mode (TIMx_CCRy). Channel 1 and channel 2 share a control register (TIMx_CCMR1), shown in Figure 7.22 and channel 3 and channel 4 share the control register TIMx_CCMR2.

Each channel can be configured to operate in input capture or output compare mode, as controlled by the mode select in CCyS in TIMx_CCMR1 or TIMx_CCMR2. The fields of these registers are dependent on the mode chosen for the channel and have a different function in input or output mode. The register fields of TIMx_CCMR1 when the channel is configured in output mode are shown in the upper row of Figure 7.22, while those for input mode are shown in the lower row.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Output	OC2CE	OC2M			OC2PE	OC2FE	CC2S		OC1CE	OC1M			OC1PE	OC1FE	CC1S	
Input	IC2F				IC2PSC		CC2S		IC1F				IC1PSC		CC1S	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7.22 TIMx_CCMR1 channel 1 and 2 control register.

The TIMxSR register holds status flags for all channels. Each channel has two flags: the CCyOF flag for overcapture, and CCyIF for interrupt. There is also the UIF flag to indicate a pending update interrupt and the TIF flag to indicate a trigger interrupt is pending. Grouping the flags in one register simplifies the code needed to identify which channel flags are set.

Input Capture Mode

Using input capture mode makes time measurement of input signals much less vulnerable to delays in processing interrupts because the channel hardware automatically captures the timer value without the need for any software intervention.

When the channel is in input capture mode, the input signal is monitored for a specific transition (rising edge, falling edge, or either). When that transition occurs, the value of the TIMx_CNT register is captured in the TIMx_CCRy register, and the channel flag CCyIF in TIMx_SR is set to one. If another consecutive capture happens before CCyIF is cleared, CCyOF is set as well to indicate an overcapture error condition. An interrupt may be generated depending on the CCyIE bit of TIMx_DIER. The ISR must copy the captured value out of the channel's TIMx_CCRy register and use it. The ISR must also clear the channel flag by writing zero to it. If the DMA field CCyDE is set in TIMx_DIER, then a DMA transfer will be requested.

We configure timer's channel as follows:

- Select input capture mode in CCyS in TIMx_CCMR1 or TIMx_CCMR2 by setting the bits to 01, 10 or 11 depending on the trigger event source.
- Optionally enable trigger filtering to remove noise. After an input trigger changes, it must be stable for this amount of time to be recognized. In the TIMx_CCMR1 or TIMx_CCMR2 register set the ICyF field to the sampling value.
- Select which edges trigger a capture using CCyNP:CCyP field in TIMx_CCER: 0:0 for rising edges, 0:1 for falling edges, 1:1 for both rising and falling edges.
- Select the input prescaler. The capture can be done after each valid transition or after 2, 4, or 8 valid transitions by setting the ICyPS bits of TIMx_CCMR1 or TIMx_CCMR2.
- Enable the capture register by setting CCyE bit in the TIMx_CCER register to one.
- If used, enable interrupts by setting CCyIE to one in the TIMx_DIER register.
- If used, enable a DMA request by setting CCyDE to one in the TIMx_DIER register.